

Unit-4: HTTP

The Hypertext Transfer Protocol (HTTP) is a standard that defines how a web client talks to a server and how data is transferred from the server back to the client. Although HTTP is usually thought of as a means of transferring HTML files and the pictures embedded in them, they can also be used to transfer Microsoft word documents, windows .exe files, or anything else that can be represented in bytes.

HTTP is the standard protocol for communication between web browsers and web servers. HTTP specifies how a client and server establish a connection, how the client requests data from the server, how the server responds to that request, and finally, how the connection is closed. HTTP connections use the TCP/IP protocol for data transfer.

For each request from client to server, there is a sequence of four steps:

1. The client opens a TCP connection to the server on port 80, by default; other ports may be specified in the URL.
2. The client sends a message to the server requesting the resource at a specified path. The request includes a header, and optionally (depending on the nature of the request) a blank line followed by data for the request.
3. The server sends a response to the client. The response begins with a response code, followed by a header full of metadata, a blank line, and the requested document or an error message.
4. The server closes the connection.

4.1 The Protocol: Keep-Alive

HTTP Keep-Alive (also known as persistent connection) is a mechanism used to keep an HTTP connection open between the client (usually a web browser) and the server, allowing multiple requests and responses to be sent over the same connection without needing to open new connections for each request. This reduces the overhead of repeatedly establishing TCP connections and thus improves performance, especially when multiple resources (e.g., images, stylesheets, and scripts) need to be loaded by the browser.

☐ How it works:

- In traditional HTTP/1.0, each request created a new TCP connection. After the request was completed, the connection would be closed.
- With Keep-Alive, the connection remains open for a certain time (default is 5-10 seconds), allowing the client to send additional requests over the same connection.

☐ Headers:

- Connection: keep-alive tells the server to keep the connection open.
- Connection: close is used to explicitly close the connection after the request.

Example:

GET /index.html HTTP/1.1

Host: www.example.com

Connection: keep-alive

4.2 HTTP Methods

HTTP defines several methods to perform different operations on resources. The most commonly used HTTP methods are:

1. GET: It retrieves data from the server (e.g., web server, images, etc.). GET is the most common HTTP request method. A client can use the GET method to request for a piece of resource from an HTTP server. A GET request message takes the following syntax:

GET request-URI HTTP-version

Example:

GET /products HTTP/1.1

Host: www.example.com

- The Keyword is case sensitive and must be in uppercase.
- Request-URI: specifies the path of resource requested, which must begin from the root “/” of the document base directory.
- HTTP-version: Either HTTP/1.0 version or HTTP/1.1. This client negotiates the protocol to be used for the current session. For example, the client may request to use HTTP/1.1. If the server does not support HTTP/1.1, it may inform the client in the response to use HTTP/1.0.
- GET request message has an optional request body which contains the query string.

Example:

```
import java.io.BufferedReader;

import java.io.InputStreamReader;

import java.net.HttpURLConnection;

import java.net.URL;
```

```
public class HttpGetExample {  
    public static void main(String[] args) throws Exception {  
        URL url = new URL("https://jsonplaceholder.typicode.com/posts/1"); // Test API  
        HttpURLConnection conn = (HttpURLConnection) url.openConnection();  
        conn.setRequestMethod("GET");  
  
        int responseCode = conn.getResponseCode();  
        System.out.println("Response Code: " + responseCode);  
  
        BufferedReader reader = new BufferedReader(new  
        InputStreamReader(conn.getInputStream()));  
        String line;  
        while ((line = reader.readLine()) != null) {  
            System.out.println(line);  
        }  
        reader.close();  
    }  
}
```

2. POST: It sends data to the server to create or modify resources (e.g., submitting a form). The data sent to the server with POST is stored in the request body of the HTTP request.

POST /submit-form HTTP/1.1

Host: www.example.com

username=johndoe&password=12345

Example:

```
import java.io.BufferedReader;
```

```
import java.io.InputStreamReader;
```

```
import java.io.OutputStream;
```

```
import java.net.HttpURLConnection;
```

```
import java.net.URL;
```

```
public class HttpPostExample {
```

```
    public static void main(String[] args) throws Exception {
```

```
        URL url = new URL("https://jsonplaceholder.typicode.com/posts"); // Test API
```

```
        HttpURLConnection conn = (HttpURLConnection) url.openConnection();
```

```
        conn.setRequestMethod("POST");
```

```
        conn.setRequestProperty("Content-Type", "application/json");
```

```
        conn.setDoOutput(true);
```

```
        // JSON data to send
```

```
        String jsonInput = "{ \"title\": \"foo\", \"body\": \"bar\", \"userId\": 1 }";
```

```
        // Write request body
```

```
try (OutputStream os = conn.getOutputStream()) {  
    os.write(jsonInput.getBytes());  
    os.flush();  
}  
  
// Read response code  
  
int responseCode = conn.getResponseCode();  
  
System.out.println("Response Code: " + responseCode);  
  
// Read and print the response body  
  
BufferedReader reader = new BufferedReader(new  
InputStreamReader(conn.getInputStream()));  
  
String line;  
  
StringBuilder response = new StringBuilder();  
  
while ((line = reader.readLine()) != null) {  
    response.append(line);  
}  
  
reader.close();  
  
System.out.println("Response Body: " + response.toString());  
}  
}
```

3. PUT: This method updates or replaces an existing resource on the server. The difference between POST and PUT is that PUT requests are idempotent (multiple requests will result in the same state). That is, calling the same PUT request multiple times will always produce the same result. In contrast, calling a POST request repeatedly have side effects of creating the same resource multiple times.

PUT /user/1 HTTP/1.1

Host: www.example.com

Content-Type: application/json

{ "name": "John", "age": 30 }

Example:

```
import java.io.OutputStream;
```

```
import java.net.HttpURLConnection;
```

```
import java.net.URL;
```

```
public class HttpPutExample {
```

```
    public static void main(String[] args) throws Exception {
```

```
        URL url = new URL("https://jsonplaceholder.typicode.com/posts/1");
```

```
        HttpURLConnection conn = (HttpURLConnection) url.openConnection();
```

```
        conn.setRequestMethod("PUT");
```

```
        conn.setRequestProperty("Content-Type", "application/json");
```

```
        conn.setDoOutput(true);
```

```
        String jsonInput = "{ \"id\": 1, \"title\": \"updated title\", \"body\": \"updated body\",  
        \"userId\": 1 }";
```

```
        try (OutputStream os = conn.getOutputStream()) {
```

```

        os.write(jsonInput.getBytes());

        os.flush();
    }

    int responseCode = conn.getResponseCode();

    System.out.println("Response Code: " + responseCode);
}
}

```

4. DELETE: The DELETE HTTP method asks the server to delete a specified resource. The DELETE method has no defined semantics for the message body, so this should be empty.

Example:

```

import java.net.HttpURLConnection;
import java.net.URL;

public class HttpDeleteExample {

    public static void main(String[] args) throws Exception {

        URL url = new URL("https://jsonplaceholder.typicode.com/posts/1");
        HttpURLConnection conn = (HttpURLConnection) url.openConnection();
        conn.setRequestMethod("DELETE");

        int responseCode = conn.getResponseCode();

        System.out.println("Response Code: " + responseCode);
    }
}

```

5. HEAD: The HTTP HEAD method is a request that asks a server for metadata about a resource, without downloading the resource itself. It is similar to the GET method, but without the body. It only retrieves headers from the server.

Example:

```
import java.net.HttpURLConnection;

import java.net.URL;

import java.util.Map;

public class HttpHeadersExample {

    public static void main(String[] args) throws Exception {

        URL url = new URL("https://www.google.com");

        HttpURLConnection conn = (HttpURLConnection) url.openConnection();

        conn.setRequestMethod("HEAD");

        int responseCode = conn.getResponseCode();

        System.out.println("Response Code: " + responseCode);

        // Print response headers
        for (Map.Entry<String, java.util.List<String>> entry : conn.getHeaderFields().entrySet()) {

            System.out.println(entry.getKey() + ": " + entry.getValue());

        }

    }

}
```

6. OPTIONS: The OPTIONS HTTP method requests permitted communication options for a given URL or server. This retrieves allowed HTTP methods from the server.

Example:

```
import java.net.HttpURLConnection;

import java.net.URI;
```



```
import java.net.URL;

public class HttpOptionsExample {

    public static void main(String[] args) throws Exception {

        URI uri = new URI("https://httpbin.org/");

        URL url = uri.toURL();

        HttpURLConnection conn = (HttpURLConnection) url.openConnection();

        conn.setRequestMethod("OPTIONS");

        // Get response code

        int responseCode = conn.getResponseCode();

        System.out.println("Response Code: " + responseCode);

        // Get allowed methods

        String allowedMethods = conn.getHeaderField("Allow");

        System.out.println("Allowed Methods: " + allowedMethods);

    }

}
```

4.3 The Request Body

The request body is the part of the HTTP request where data is sent to the server. It is primarily used in methods like POST, PUT, and PATCH, where the client needs to send data (e.g., form data, JSON payload) to the server for processing or storage. The GET method retrieves a representation of a resource identified by a URL. The exact location of the resource you want to GET from a server is specified by the various parts of the path and query string. How different paths and query strings map to different resources is determined by the server.

POST and PUT are more complex. In these cases, the client supplies the representation of the resource, in addition to the path and the query string. The representation of the resource is sent in the body of the request, after the header. That is, it sends these four items in order:

1. A starter line including the method, path and query string, and HTTP version.
2. An HTTP header.
3. A blank line (two successive carriage return/linefeed pairs).
4. The body

For example, this POST request sends form data to server:

```
POST /cgi-bin/register.pl HTTP 1.0
```

```
Date: Thursday, 19 Feb 2025, 11:21:34
```

```
Host: www.cafeaulait.org
```

```
Content-type: application/x-www-form-urlencoded
```

```
Content-length: 54
```

In this example, the body contains an application/x-www-form-urlencoded data, but that's just one possibility. In general, the body can contain arbitrary bytes. However, the HTTP header should include two fields that specify the nature of the body:

- A Content-length field that specifies how many bytes are in the body (54 in the example).
- A Content-type field that specifies the MIME media type of the bytes (application/x-www-form-urlencoded in the example).

Example of sending JSON in the body:

```
POST /api/users HTTP/1.1
```

```
Host: www.example.com
```

```
Content-Type: application/json
```

```
{  
  "username": "johndoe",  
  "email": "johndoe@example.com"  
}
```

4.4 Cookies: CookieManager and CookiesStore

Cookies are small pieces of data stored by the browser on the client's side. They are used for session management, personalization, tracking, etc. In HTTP, cookies are sent as part of the HTTP request and response headers.

CookieManager: In Java, CookieManager is used to manage cookies automatically. It allows you to store cookies in HTTP responses and send them with subsequent requests. It maintains cookies across different sessions and allows you to handle cookie storage and retrieval automatically.

- **Default CookieManager:** Used automatically by Java when making HTTP requests.

Example:

```
CookieManager cookieManager = new CookieManager();  
CookieHandler.setDefault(cookieManager);
```

- **Custom CookieManager:** Allows you to control how cookies are stored and retrieved.

Example:

```
CookieManager customCookieManager = new CookieManager(null,  
CookiePolicy.REJECT_ALL);  
CookieHandler.setDefault(customCookieManager);
```

CookiePolicy provides some pre-defined implementations for convenience:

- `CookiePolicy.ACCEPT_ORIGINAL_SERVER` – Only cookies from the original server can be saved (the default implementation).
- `CookiePolicy.ACCEPT_ALL` – all cookies can be saved regardless of their origin.
- `CookiePolicy.REJECT_ALL` – no cookies can be saved.

CookiesStore: This is an interface used by CookieManager to store and manage the cookies. It allows adding, retrieving and managing cookies. It's useful for managing cookies across multiple requests for a specific domain.

Example:

```
CookieStore cookieStore = cookieManager.getCookieStore();
```

```
List<HttpCookie> cookies = cookieStore.getCookies();
```

Cookies in HTTP headers:

- **Set-Cookie:** Used by the server to send cookies to the client.
- **Cookie:** Used by the client to send cookies back to the server.

*Program to store and retrieve cookies.

```
import java.io.BufferedReader;
```

```
import java.io.InputStreamReader;
```

```
import java.net.*;
```

```
import java.util.List;
```

```
public class StoreRetrieveCookies {
```

```
    public static void main(String[] args) throws Exception {
```

```
        // Custom CookieManager that accepts all cookies
```

```
        CookieManager customCookieManager = new CookieManager(null,  
CookiePolicy.ACCEPT_ALL);
```

```
        CookieHandler.setDefault(customCookieManager);
```

```
        // Request to set cookies
```

```
        URL url = new URL("https://httpbin.org/cookies/set?username=JohnDoe");
```

```
        HttpURLConnection conn = (HttpURLConnection) url.openConnection();
```

```
        conn.setRequestMethod("GET");
```

```
        // Read response
```

```
        BufferedReader in = new BufferedReader(new  
InputStreamReader(conn.getInputStream()));
```

```
        while (in.readLine() != null) {} // Ignore content
```

```
        in.close();
```

```
        // Retrieve stored cookies
```

```
        CookieStore cookieStore = customCookieManager.getCookieStore();
```

```
        List<HttpCookie> cookies = cookieStore.getCookies();
```

```
        System.out.println("Stored Cookies:");
```

```
        for (HttpCookie cookie : cookies) {
```

```
        System.out.println(cookie);
    }
}
}
```

*Program to retrieve cookie information stored in the system.

```
import java.net.*;
```

```
import java.util.List;
```

```
public class RetrieveCookiesExample {
    public static void main(String[] args) throws Exception {
        // Set up a CookieManager to handle cookies
        CookieManager cookieManager = new CookieManager();
        CookieHandler.setDefault(cookieManager);

        // Send a request to a website that sets cookies
        URL url = new URL("https://httpbin.org/cookies/set?user=JohnDoe&session=abc123");
        HttpURLConnection conn = (HttpURLConnection) url.openConnection();
        conn.setRequestMethod("GET");
        conn.getResponseCode(); // Trigger the request

        // Retrieve stored cookies
        CookieStore cookieStore = cookieManager.getCookieStore();
        List<HttpCookie> cookies = cookieStore.getCookies();

        // Display cookies
        System.out.println("Stored Cookies:");
        if (cookies.isEmpty()) {
```

```
        System.out.println("No cookies stored.");
    } else {
        for (HttpCookie cookie : cookies) {
            System.out.println("Name: " + cookie.getName());
            System.out.println("Value: " + cookie.getValue());
            System.out.println("Domain: " + cookie.getDomain());
            System.out.println("Path: " + cookie.getPath());
            System.out.println("Max Age: " + cookie.getMaxAge());
            System.out.println("Secure: " + cookie.getSecure());
            System.out.println("-----");
        }
    }
}
}
```